



UNIVERSIDAD  
**NACIONAL**  
DE COLOMBIA

**Universidad Nacional de Colombia - sede Bogotá**  
**Facultad de Ingeniería**  
**Departamento de Sistemas e Industrial**  
**Curso: Ingeniería de Software 1 (2016701)**

### **Tutorial App desde cero**

El objetivo de este documento es dar una guía detallada de como levantar una APP desde cero. Se va a trabajar con Python para el Backend, usando como framework Django y la Base de Datos en PostgreSQL. La app se va a levantar con la ayuda de contenedores en Docker

1. Instalar Docker Desktop
2. Puede que pida actualizar la versión de linux para tu sistema operativo, para windows es ejecutar el comando en la terminal → [wsl --update](#)
3. Para mantener un orden recomendamos una organización de directorios de este estilo

mi-proyecto-gym/

```
|— backend/          <-- Aquí va Django (Python)
|   |— manage.py
|   |— core/         <-- Configuración (settings.py)
|   |— gym_app/      <-- Tus modelos y lógica
|   |— Dockerfile    <-- Instrucciones para empaquetar Python
|   |— requirements.txt
|— frontend/        <-- (A futuro, para React/Vue/etc.)
|— docker-compose.yml <-- El "director de orquesta"
|— .gitignore
```

4. El archivo **docker-compose.yml** se va a encargar de darle la configuración al contenedor para levantar la base de datos en postgresQL, también se podría realizar directamente en la interfaz de Docker, pero esto hace que cada vez que se quiera ejecutar el contenedor toque crearlo y configurarlo de cero.

```
services:
  db:
    image: postgres:15
    container_name: postgres_gym
    environment:
      POSTGRES_DB: gym_db      # Nombre de la base de datos
      POSTGRES_USER: admin_gym # Usuario
```

```
POSTGRES_PASSWORD: password # Contraseña
ports:
  - "5432:5432" # Conecta el puerto de Docker con PC
volumes:
  - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data: # Esto evita que los datos se borren al apagar el
contenedor
```

5. Para finalmente levantarlo, ubicado en la carpeta del proyecto ejecutar en terminal `docker-compose up -d`
6. Para confirmar en la interfaz de Docker, en la sección de Containers debe aparecer el contenedor en estado running
7. Ahora vamos a crear la conexión con Django (Nuestro framework), lo primero es tener el archivo `requirements.txt` que se encarga de almacenar las librerías que se van a instalar. Por el momento

django

psycopg2-binary #librería de ORM para que Python hable con PostgreSQL

8. Ahora creamos un archivo sin extensión llamado Dockerfile, este se encarga de decirle a docker como empaquetar el código de Python
9. Ahora en la raíz del proyecto ejecutamos esto: `docker-compose run web django-admin startproject core` . , con el fin de generar los archivos automáticos de Django.

Lo más probable es que ocurra un error porque aún no se ha presentado el servicio web dentro de `docker-compose.yml`. Consiste en empaquetar tu aplicación y sus dependencias en un contenedor aislado, esto garantiza que funcione exactamente igual en cualquier entorno, ya sea tu máquina local o un servidor en la nube

10. Para solucionarlo actualizamos docker-compose

```
services:
  db:
    image: postgres:15
    container_name: postgres_gym
    environment:
      POSTGRES_DB: gym_db # Nombre de la base de datos
```

```

POSTGRES_USER: admin_gym    # Usuario
POSTGRES_PASSWORD: password # Contraseña
ports:
  - "5432:5432"             # Conecta el puerto de Docker con PC
volumes:
  - postgres_data:/var/lib/postgresql/data
web:
  build: ./backend          # Docker buscará el Dockerfile dentro de la carpeta
                              /backend
  container_name: django_gym
  command: python manage.py runserver 0.0.0.0:8000
  volumes:
    - ./backend:/code       # Sincroniza el código local con el contenedor
  ports:
    - "8000:8000"
  environment:
    - DATABASE_NAME=gym_db
    - DATABASE_USER=admin_gym
    - DATABASE_PASSWORD=password
    - DATABASE_HOST=db
  depends_on:
    - db                    # Esto le dice a Docker que espere a que la BD esté lista
  volumes:
    postgres_data:         # Esto evita que los datos se borren al apagar el
                              contenedor

```

Ahora construimos el servicio web a través de la terminal `docker-compose build web`, para ahora si ejecutar el comando que cargue los archivos de Django `docker-compose run web django-admin startproject core`.

Para confirmar se debió crear la carpeta core y el archivo manage.py dentro de backend

11. Ahora se debe modificar al archivo `settings.py` que fue creado en la carpeta core, se busca la sección DATABASES y se cambia por los siguiente:

```

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'gym_db',      # Nombre que pusiste en el YAML
        'USER': 'admin_gym',   # Usuario del YAML

```

```
'PASSWORD': 'password', # Contraseña del YAML
'HOST': 'db',           # Nombre del servicio en el YAML
'PORT': '5432',
}
```

Si analizamos, es la misma información que se configuró en el archivo `docker-compose`

12. En Django, el código se organiza en "apps". Hacemos una llamada `gym_app` para agregar los modelos `docker-compose exec web python manage.py startapp gym_app`.

Si por alguna razón dice que el servicio web no está corriendo puede deberse a que el comando `startproject` intentó crear archivos que ya existían. Para solucionarlo nos aseguramos de que no haya procesos zombies. Ejecutamos los comandos en terminal

```
docker-compose down
docker-compose up -d
```

Finalmente verificamos que en la interfaz de Docker el contenedor `django_gym` (en el caso de este ejemplo), este corriendo

13. Ahora nuevamente en `settings.py` debemos agregar esta aplicación 'gym\_app' al apartado `INSTALLED_APPS`

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'gym_app',
]
```

14. Ahora vamos a definir los modelos, "instanciamos las entidades" traduciendo tu esquema DBML a clases de Python. Por fines prácticos se usará un ejemplo básico ya definido en la estructura de la DB. En el directorio de `gym_app`, buscamos `models.py`, este será el espacio para crear todas nuestras entidades.

```
from django.db import models
```

```
class Goal(models.Model):
    name = models.CharField(max_length=100)
    description = models.TextField()

    def __str__(self):
        return self.name

class UserProfile(models.Model): # Evitamos llamarla 'User' para no chocar
con el sistema de Django
    name = models.CharField(max_length=255)
    age = models.IntegerField()
    weight = models.FloatField()
    goal = models.ForeignKey(Goal, on_delete=models.CASCADE)

    def __str__(self):
        return self.name
```

15. Ahora vamos con el paso donde la ORM lee las clases de python y crea las tablas de la base de datos

Primero ejecutamos en terminal `docker-compose exec web python manage.py makemigrations gym_app`, esto nos debe dar como respuesta la creación de los modelos

Para luego ejecutar `docker-compose exec web python manage.py migrate`, esto aplica los cambios sobre la base de datos

16. Ahora debemos hacer el “Hola Mundo” de Backend+DB, el cual probaremos con Postman. Pero para poder hacer esto debemos crear una vista que devuelve un .json, ya que este funciona principalmente como una Colección (que guarda todas tus peticiones y rutas de la API) o como un Entorno (que guarda variables como la URL base o los tokens de autenticación).

17. En el directorio gym\_app buscamos el archivo views.py, creamos la vista:

```
from django.http import JsonResponse
from .models import Goal

def prueba_conexion(request):
    # Creamos un dato de prueba en la BD
    meta, created = Goal.objects.get_or_create(
```

```

name="Ganar masa muscular",
defaults={'description': 'Aumentar volumen mediante hipertrofia'}
)

return JsonResponse({
    "status": "¡Conexión Exitosa!",
    "database": "PostgreSQL",
    "dato_recuperado": {
        "id": meta.id,
        "meta": meta.name
    }
})

```

18. Ahora configuramos la ruta para poder acceder desde el navegador o Postman:

```

from django.contrib import admin
from django.urls import path
from gym_app.views import prueba_conexion

urlpatterns = [
    path('admin/', admin.site.urls),
    path('test/', prueba_conexion),
]

```

19. El siguiente paso es hacer la prueba a través de Postman, se recomienda instalar la versión de escritorio, pero también hay una extensión para VScode. Para este caso usamos. Para validar el Hola mundo, dentro de Postman creamos un nuevo request, nos aseguramos que esté en HTTP, seleccionamos GET, para luego ingresar la ruta <http://localhost:8000/test/> y enviamos

20. Lo que deberíamos observar es en el body de Postman algo como esto

```

{
  "status": "¡Conexión Exitosa!",
  "database": "PostgreSQL",
  "dato_recuperado": {
    "id": 1,
    "meta": "Ganar masa muscular"
  }
}

```

```
}  
}
```

Esta prueba confirma que se cumplió con todos los puntos del ejercicio:

- Conexión a la BD: Django logró hablar con Postgres.
- Instanciación de Entidades: El ORM creó y recuperó un objeto de la clase **Goal** que se definió basándose en el esquema de DB de este ejemplo.
- Funcionamiento del Backend: El servidor de Python en Docker está procesando solicitudes correctamente.